ST2195 COURSEWORK REPORT

PROGRAMMING FOR DATA SCIENCE

MARGARET JANELLE KAMTO
220657356

Table of Contents

Introduction 3

1.a Markov Chain Monte Carlo 4

1.b $\widehat{R}$ Convergence Analysis 4

Introduction

The Metropolis-Hastings algorithm of Markov Chain Monte Carlo (MCMC) is used in the first part to simulate a probability density function using random walks. This was calculated initially using several preset parameters, with 10000 generated samples and an SD of 1. By generating a uniform random number between 0-1, comparing it to the ratio and appending it to the array if certain criteria is met, it has been able to accurately approximate the true density function by utilizing this technique. $\hat{R}$ scoring is also used to perform a convergence analysis, which determines whether a particular algorithm reaches its convergence point. This was done by generating 4 sample group and calculating each mean and variance for each sample group, then using the overall sample mean and variance, the R score can be calculated. The closer R is to 1, the stronger the indication that the function converges, and vice versa

1.a Markov Chain Monte Carlo

The function is defined as $f(x) = \frac{1}{2}\exp(-|x|)$ where this equation shows the target probability density function which can be modeled in Python and R using the following code:

Py:

```
def f(x):
    return 0.5 * np.exp(-np.abs(x))
```

R:

```
f <- function(x) {
  return(0.5 * exp(-abs(x)))
}
```

Then, a random walk metropolis algorithm is applied using N = 10000 and s = 1 where N is the number of samples and s is the standard deviation for the random number generator. The algorithm also accepts a number as the initial value x_0. This algorithm starts by generating the value of x_* (x_star) randomly using a normal distribution with mean x_0 (2 in this case) and standard deviation s. Then, the log ratio is calculated using this following equation:

$$\log r(x_*, x_{i-1}) = \log f(x_*) - \log f(x_{i-1})$$

Where x_i is the i-th randomly generated number from the generation algorithm.

Then a random uniform number u is generated from 0 and 1 and is checked whether $\log u < \log r$ and if it does, append the value of x_* to the sample result, else, append the value of x_{i-1} which can be modeled with this equation:

$$x_i = \begin{cases} x_* & , \log u < \log r \\ x_{i-1} & , \log u \geq \log r \end{cases}$$

When plotted using histogram (Fig. 1) against the actual density function, The graph closely resembles the probability density function distribution even after going through several randomization processes. The sample mean and standard deviation are as follows:

Sample mean: 0.03920569539238854

Sample standard deviation: 1.3971550102271173

1.b $\hat{R}$ Convergence Analysis

Using R value, the convergence of a function can be calculated. By using N=2000 with 4 generated sequence J, the R value can be calculated by using the equation of $\hat{R} = \sqrt{\frac{B+W}{W}}$ where B is the **between sample variance** and W is the **within sample variance**. Iterating the standard deviation value s between 0.01 to 1, The value of R approaches 1 (Fig. 2), signaling that the random walk algorithm successfully converges.

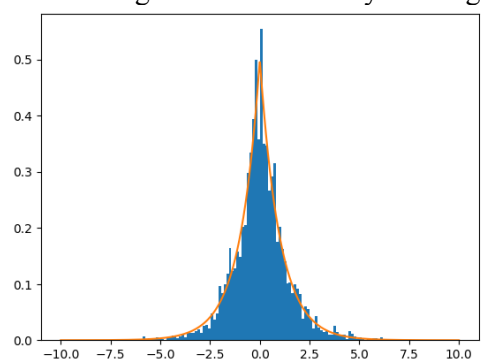

Figure1 f(x) estimate using generated samples (Py)

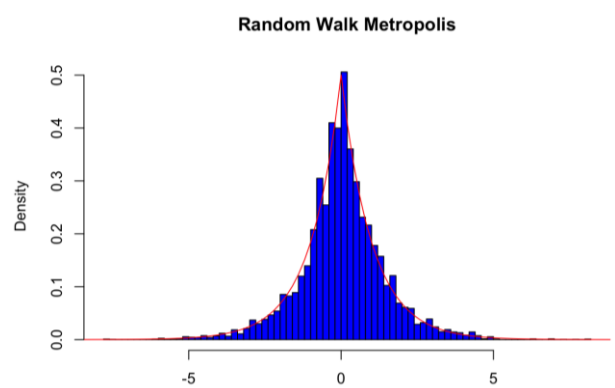

Figure3 f(x) estimate using generated samples (R)

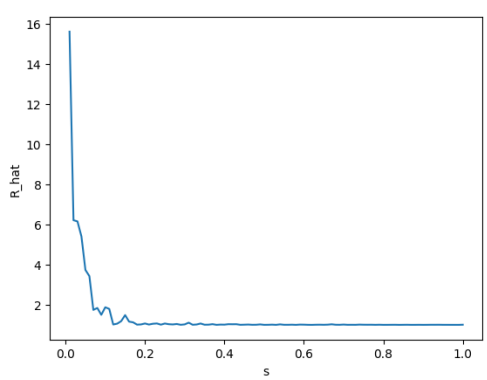

Figure2 Rhat value over s from 0.01 to 1 (Py)

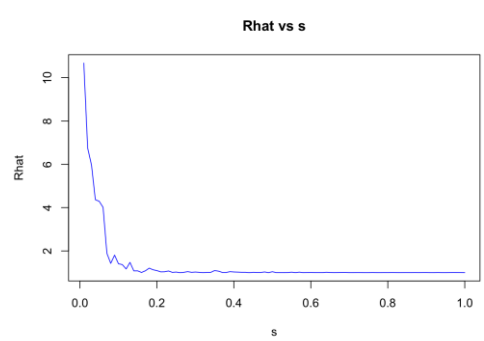

Figure4 Rhat value over s from 0.01 to 1 (R)